

Stripe Setup Guide für Qryx

Status: Setup Required

Geschätzte Zeit: 15-20 Minuten

Voraussetzung: Stripe Account (<https://stripe.com>)

Übersicht

Dieser Guide führt dich durch die komplette Stripe-Konfiguration für Qryx:

1.  Stripe Account erstellen/vorbereiten
 2.  Produkte & Preise erstellen
 3.  Webhook konfigurieren
 4.  API Keys abrufen
 5.  Environment Variables setzen
 6.  Testen
-

Schritt 1: Stripe Dashboard öffnen

Test Mode vs. Live Mode

Für Entwicklung & Testing:

- Verwende **Test Mode** (Toggle oben rechts)
- URL: <https://dashboard.stripe.com/test>
- Test Cards funktionieren nur im Test Mode

Für Production:

- Verwende **Live Mode**
- URL: <https://dashboard.stripe.com>
- Echte Kreditkarten erforderlich

 **Tipp:** Starte mit Test Mode, migriere später zu Live Mode

Schritt 2: Produkte erstellen

2.1 Zum Product Catalog navigieren

1. Öffne: <https://dashboard.stripe.com/test/products>
2. Klicke: "+ Add product"

2.2 Produkt 1: Qryx Starter

Produktinformationen:

Name: Qryx Starter
 Description: Perfect **for** small businesses getting started **with** AI sales
 Statement descriptor: QRYX STARTER (appears on customer's credit card)

Pricing:

Pricing model: Standard pricing
 Price: \$29.00 USD
 Billing period: Monthly
 Payment **type**: Recurring

Features (optional):

- 1,000 AI conversations per month
- Basic product recommendations
- Email support
- 7-day free trial

Nach dem Erstellen:

1. Klicke auf das erstellte Produkt
2. Kopiere die **Price ID** (beginnt mit `price_...`)
3. Speichere sie als: `STRIPE_PRICE_STARTER`

Beispiel:

`price_1QryxStarterTestABC123`

2.3 Produkt 2: Qryx Professional ★ (Most Popular)

Produktinformationen:

Name: Qryx Professional
 Description: Advanced AI sales assistant **for** growing businesses
 Statement descriptor: QRYX PRO

Pricing:

Pricing model: Standard pricing
 Price: \$99.00 USD
 Billing period: Monthly
 Payment **type**: Recurring

Features (optional):

- 10,000 AI conversations per month
- Advanced product recommendations
- Custom branding
- Priority email & chat support
- Analytics dashboard
- 14-day free trial

Nach dem Erstellen:

1. Klicke auf das erstellte Produkt
2. Kopiere die **Price ID**
3. Speichere sie als: `STRIPE_PRICE_PROFESSIONAL`

Beispiel:

```
price_1QRyxProfessionalTestXYZ456
```

2.4 Produkt 3: Qryx Enterprise

Produktinformationen:

```
Name: Qryx Enterprise
Description: Custom AI solution for enterprise organizations
Statement descriptor: QRYX ENTERPRISE
```

Pricing:

```
Pricing model: Contact sales (no fixed price)
OR
Price: $499.00 USD (if you want a starting price)
Billing period: Monthly
Payment type: Recurring
```

Features (optional):

- Unlimited AI conversations
- White-label solution
- Dedicated account manager
- Custom integrations
- SLA guarantee
- Phone support
- Custom contract

Nach dem Erstellen:

1. Kopiere die **Price ID** (falls du einen Preis gesetzt hast)
2. Speichere sie als: `STRIPE_PRICE_ENTERPRISE`



Hinweis: Enterprise kann auch als “Contact Sales” Button ohne direkten Stripe Checkout konfiguriert werden.



Schritt 3: Webhook konfigurieren

3.1 Webhook Endpoint erstellen

1. Öffne: <https://dashboard.stripe.com/test/webhooks>
2. Klicke: **“Add endpoint”**

3.2 Endpoint URL eingeben

Für Production (Vercel):

```
https://www.jnxfabs.ai/api/stripe/webhook
```

Für Local Development:

```
http://localhost:3000/api/stripe/webhook
```

 **Wichtig:** Für lokales Testen musst du den Stripe CLI verwenden (siehe unten)

3.3 Events auswählen

Wähle **genau diese Events**:

- ☒ checkout.session.completed
- ☒ customer.subscription.updated
- ☒ customer.subscription.deleted
- ☒ invoice.payment_succeeded
- ☒ invoice.payment_failed

Was jedes Event macht:

Event	Beschreibung	Verwendung
checkout.session.completed	Zahlung abgeschlossen	Subscription erstellen
customer.subscription.updated	Plan geändert	Subscription updaten
customer.subscription.deleted	Subscription gekündigt	Status auf "canceled" setzen
invoice.payment_succeeded	Monatliche Zahlung erfolgreich	Subscription erneuern
invoice.payment_failed	Zahlung fehlgeschlagen	Status auf "past_due" setzen

3.4 Signing Secret kopieren

Nach dem Erstellen:

1. Klicke auf den Endpoint
2. Scrolle zu **"Signing secret"**
3. Klicke **"Reveal"**
4. Kopiere den Secret (beginnt mit `whsec_...`)
5. Speichere ihn als: `STRIPE_WEBHOOK_SECRET`

Beispiel:

```
whsec_1234567890abcdefghijklmnopqrstuvwxyz
```

Schritt 4: API Keys abrufen

4.1 API Keys Seite öffnen

1. Öffne: <https://dashboard.stripe.com/test/apikeys>
2. Du siehst zwei Arten von Keys:

4.2 Secret Key (Server-side)

Publishable Key:

Location: "Standard keys" → "Publishable key"
Format: pk_test_...
Usage: Client-side (öffentlich sichtbar)

1. Kopiere den **Publishable key**
2. Speichere ihn als: `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`

Beispiel:

```
pk_test_51JXyzABC123def456GHI789jklMN0012pqrSTU345vwxyz678
```

4.3 Publishable Key (Client-side)

Secret Key:

Location: "Standard keys" → "Secret key" → "Reveal test key"
Format: sk_test_...
Usage: Server-side only (GEHEIM!)

1. Klicke **"Reveal test key"**
2. Kopiere den **Secret key**
3. Speichere ihn als: `STRIPE_SECRET_KEY`

Beispiel:

```
sk_test_51JXyzABC123def456GHI789jklMN0012pqrSTU345vwxyz678901abcDEF234ghiJKL567mnoP890
```

 **WARNUNG:** Der Secret Key darf NIEMALS im Client-Code oder in Git committed werden!

Schritt 5: Session Secret generieren

5.1 Session Secret erstellen

Der `SESSION_SECRET` wird für die verschlüsselte Shop-Session verwendet.

Terminal öffnen und ausführen:

```
openssl rand -base64 32
```

Output (Beispiel):

```
ABCdefGHI123jklMN0456pqrSTU789vwxyz012=
```

Speichere diesen Wert als: `SESSION_SECRET`



Schritt 6: Environment Variables zusammenstellen

6.1 Alle gesammelten Werte

Du solltest jetzt folgende Werte haben:

```
# Stripe API Keys
STRIPE_SECRET_KEY=sk_test_...
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test_...

# Webhook Secret
STRIPE_WEBHOOK_SECRET=whsec_...

# Price IDs
STRIPE_PRICE_STARTER=price_...
STRIPE_PRICE_PROFESSIONAL=price_...
STRIPE_PRICE_ENTERPRISE=price_...

# Shop Session Encryption
SESSION_SECRET=<generiert mit openssl>
```

6.2 In .env eintragen (Lokal)

```
cd /home/ubuntu/jnx-os/nextjs_space
nano .env
```

Füge die Variablen hinzu (oder update bestehende).

6.3 In Vercel eintragen (Production)

Option A: Vercel Dashboard

1. Gehe zu: [https://vercel.com/\[dein-projekt\]/settings/environment-variables](https://vercel.com/[dein-projekt]/settings/environment-variables)
2. Füge jede Variable einzeln hinzu:
 - Name: `STRIPE_SECRET_KEY`
 - Value: `sk_test_...`
 - Environment: Production, Preview, Development
3. Wiederhole für alle Variablen

Option B: Vercel CLI

```
vercel env add STRIPE_SECRET_KEY
# Wert eingeben wenn gefragt
# Environment auswählen: Production, Preview, Development
```

Schritt 7: Database Schema erstellen

7.1 Supabase SQL Editor öffnen

1. Gehe zu: [https://supabase.com/dashboard/project/\[dein-projekt\]/editor](https://supabase.com/dashboard/project/[dein-projekt]/editor)
2. Öffne den SQL Editor

7.2 SQL ausführen

Kopiere und führe aus:

```

-- Create billing_customers table if not exists
CREATE TABLE IF NOT EXISTS billing_customers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id TEXT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  org_id UUID REFERENCES orgs(id) ON DELETE CASCADE,
  stripe_customer_id TEXT UNIQUE,
  stripe_subscription_id TEXT UNIQUE NOT NULL,
  plan_id TEXT NOT NULL,
  plan_name TEXT NOT NULL,
  status TEXT NOT NULL CHECK (status IN ('active', 'trialing', 'past_due',
'canceled', 'incomplete', 'incomplete_expired', 'unpaid')),
  current_period_start TIMESTAMPTZ,
  current_period_end TIMESTAMPTZ,
  cancel_at_period_end BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Create indexes for faster lookups
CREATE INDEX IF NOT EXISTS idx_billing_user_id
  ON billing_customers(user_id);

CREATE INDEX IF NOT EXISTS idx_billing_stripe_subscription_id
  ON billing_customers(stripe_subscription_id);

CREATE INDEX IF NOT EXISTS idx_billing_status
  ON billing_customers(status);

CREATE INDEX IF NOT EXISTS idx_billing_org_id
  ON billing_customers(org_id);

-- Create updated_at trigger
CREATE OR REPLACE FUNCTION update_billing_customers_updated_at()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = NOW();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER billing_customers_updated_at
  BEFORE UPDATE ON billing_customers
  FOR EACH ROW
  EXECUTE FUNCTION update_billing_customers_updated_at();

-- Verify table structure
SELECT
  column_name,
  data_type,
  is_nullable
FROM information_schema.columns
WHERE table_name = 'billing_customers'
ORDER BY ordinal_position;

```

7.3 Verification

Erwartete Columns:

id	uuid	NO
user_id	text	NO
org_id	uuid	YES
stripe_customer_id	text	YES
stripe_subscription_id	text	NO
plan_id	text	NO
plan_name	text	NO
status	text	NO
current_period_start	timestampz	YES
current_period_end	timestampz	YES
cancel_at_period_end	boolean	YES
created_at	timestampz	YES
updated_at	timestampz	YES

✓ Wenn alle Columns vorhanden sind, ist die Tabelle bereit!

Schritt 8: Testen

8.1 Lokales Testing mit Stripe CLI

Stripe CLI installieren:

```
# macOS
brew install stripe/stripe-cli/stripe

# Windows
scoop install stripe

# Linux
curl -s https://packages.stripe.com/api/security/keypair/stripe-cli-gpg/public | gpg -
-dearmor | sudo tee /usr/share/keyrings/stripe.gpg
echo "deb [signed-by=/usr/share/keyrings/stripe.gpg] https://packages.stripe.com/
stripe-cli-debian-local stable main" | sudo tee -a /etc/apt/sources.list.d/stripe.list
sudo apt update
sudo apt install stripe
```

Stripe CLI mit Account verbinden:

```
stripe login
```

Webhook forwarding starten:

```
# Terminal 1: Dev Server
cd /home/ubuntu/jnx-os/nextjs_space
yarn dev

# Terminal 2: Stripe CLI
stripe listen --forward-to localhost:3000/api/stripe/webhook
```

Der CLI gibt dir einen **Webhook Signing Secret** für lokales Testing:

```
Ready! Your webhook signing secret is whsec_...
```

Update deine lokale `.env` mit diesem Secret:

```
STRIPE_WEBHOOK_SECRET=whsec_<von_stripe_cli>
```

8.2 Test Flow durchführen

Schritt 1: Installation starten

```
http://localhost:3000/api/qryx/install?shop=shopbotv3.myshopify.com
```

Schritt 2: Login/Signup

- Verwende deine Test-Credentials oder erstelle einen neuen Account

Schritt 3: Pricing Page

- Du solltest auf `/products/qryx/setup` landen
- Siehst du alle 3 Pricing Plans?

Schritt 4: Checkout

- Klicke auf "Subscribe Now" (z.B. Professional)
- Stripe Checkout öffnet sich

Schritt 5: Test Payment

Verwende diese Test Cards:

Card Number	Verhalten
4242 4242 4242 4242	✓ Erfolgreiche Zahlung
4000 0000 0000 0002	✗ Zahlung abgelehnt
4000 0025 0000 3155	🔒 3D Secure erforderlich

Card Details:

```
Card Number: 4242 4242 4242 4242
Expiry: 12/34 (beliebiges zukünftiges Datum)
CVC: 123
ZIP: 12345
```

Schritt 6: Nach erfolgreicher Zahlung

- Du wirst zu `/api/qryx/start-oauth` redirected
- Dann zu Shopify OAuth
- Nach Approval zu `/app/qryx`
- Status sollte "Connected" sein

Schritt 7: Subscription in DB prüfen

```
SELECT
  user_id,
  plan_name,
  status,
  stripe_subscription_id,
  current_period_end
FROM billing_customers
ORDER BY created_at DESC
LIMIT 5;
```

✓ Du solltest deine Subscription sehen!

8.3 Webhook Logs prüfen

Im Stripe CLI Terminal solltest du sehen:

```
✓ webhook: checkout.session.completed
✓ webhook: customer.subscription.created
✓ webhook: invoice.payment_succeeded
```

In Stripe Dashboard:

1. Gehe zu: <https://dashboard.stripe.com/test/events>
2. Prüfe die letzten Events
3. Klicke auf `checkout.session.completed`
4. Scrolle zu "Webhook attempts"
5. Status sollte "Succeeded" sein

✓ Checkliste: Setup Complete

Markiere ab, was du erledigt hast:

Stripe Dashboard

- ☐ Test Mode aktiviert
- ☐ Produkt "Qryx Starter" erstellt (\$29)
- ☐ Produkt "Qryx Professional" erstellt (\$99)
- ☐ Produkt "Qryx Enterprise" erstellt (Custom)
- ☐ Alle 3 Price IDs kopiert
- ☐ Webhook Endpoint erstellt
- ☐ 5 Events ausgewählt
- ☐ Webhook Secret kopiert
- ☐ Publishable Key kopiert
- ☐ Secret Key kopiert

Environment

- ☐ `SESSION_SECRET` generiert
- ☐ Alle 7 Environment Variables in `.env`
- ☐ Alle 7 Environment Variables in Vercel
- ☐ Vercel neu deployed (damit neue Env Vars aktiv sind)

Database

- [] `billing_customers` Tabelle erstellt
- [] Indexes erstellt
- [] Trigger für `updated_at` erstellt
- [] Tabelle verifiziert (13 columns)

Testing

- [] Stripe CLI installiert
- [] `stripe login` ausgeführt
- [] Webhook forwarding getestet
- [] Installation Flow getestet
- [] Test Card Zahlung erfolgreich
- [] Subscription in DB sichtbar
- [] Webhook Events empfangen



Troubleshooting

Problem: Webhook nicht empfangen

Symptom:

- Zahlung erfolgreich, aber keine Subscription in DB
- "Subscription pending" Error

Lösung:

```
# 1. Prüfe Webhook Secret
echo $STRIPE_WEBHOOK_SECRET

# 2. Prüfe Stripe Dashboard → Webhooks → Events
# Siehst du "Failed" attempts?

# 3. Prüfe API Logs in Vercel
# Gehe zu: https://vercel.com/[projekt]/logs
# Filter: /api/stripe/webhook

# 4. Test lokal mit Stripe CLI
stripe listen --forward-to localhost:3000/api/stripe/webhook
```

Problem: "Invalid Price ID"

Symptom:

- Fehler beim Klick auf "Subscribe Now"

Lösung:

```
# 1. Prüfe Environment Variables
echo $STRIPE_PRICE_STARTER
echo $STRIPE_PRICE_PROFESSIONAL

# 2. Prüfe, ob sie mit "price_" beginnen

# 3. Vercel: Redeploy nach Env Var Änderung
vercel --prod

# 4. Restart dev server
yarn dev
```

Problem: “Shop session expired”

Symptom:

- Nach Zahlung: “Shop session expired, please restart installation”

Ursache:

- Nutzer hat >30 Minuten zwischen Installation und Zahlung gebraucht

Lösung:

- Installation neu starten
- Oder: Session Expiry in `lib/session/shop-session.ts` erhöhen:

```
const SESSION_EXPIRY = 60 * 60; // 1 Stunde statt 30 Min
```

Problem: “Subscription pending”

Symptom:

- Nach Zahlung: “Waiting for subscription”

Ursache:

- Webhook noch nicht verarbeitet (1-3 Sekunden)

Lösung:

- Warte 3 Sekunden und refreshe
- Oder klicke “Try Again”
- Wenn >10 Sekunden: Prüfe Webhook Logs



Nächste Schritte

Nach erfolgreichem Setup:

1. Production Deployment:

```
bash
cd /home/ubuntu/jnx-os
git add .
git commit -m "Add Stripe billing integration"
git push origin main
```

2. Vercel Auto-Deploy:

- Prüfe Deploy Status: [https://vercel.com/\[projekt\]](https://vercel.com/[projekt])
- Warte auf “Production” Status

3. Production Testing:

- Teste mit echtem Shopify Store
- Verwende Test Mode zuerst
- Dann migriere zu Live Mode

4. Live Mode Migration:

- Erstelle Produkte in Live Mode
- Update Webhook zu Live Mode
- Replace alle Env Vars mit Live Keys
- Test mit echter Kreditkarte

5. Phase 5B starten:

- Billing Dashboard UI (`/app/billing`)
- Usage Tracking
- Email Notifications
- Admin Tools

Support

Stripe Support:

- Dashboard: <https://dashboard.stripe.com>
- Docs: <https://stripe.com/docs>
- Support: <https://support.stripe.com>

JNX Support:

- Documentation: `/home/ubuntu/jnx-os/PHASE5A_COMPLETION_SUMMARY.md`
- Quick Reference: `/home/ubuntu/jnx-os/PHASE5A_QUICK_REFERENCE.md`

Version: 1.0

Last Updated: 2024-12-29

Status: Ready for Production Testing 🚀